HW2 Computer Vision

Rashin Rahnamoun

400243092

1 Q1.Convolution Neural Network for Image Classification

1.1 Introduction

Image classification is a fundamental task in computer vision that assigns an image to one of several predefined categories. This report focuses on the classification of shoe images using a convolutional neural network (CNN) and describes the underlying mathematical principles and algorithms.

1.2 Data Preparation

To prepare the dataset, the images are first scaled to a range of $[0, 1]$ by normalizing the pixel values:

$$x' = \frac{x}{255}, \tag{1}$$

where x is the original pixel value and x' is the normalized value. This rescaling ensures consistent input data for the model.

The images are resized to a fixed dimension of 150×150 pixels to maintain uniformity across the dataset, and augmentation techniques are applied to enhance model generalization.

1.3 Model Architecture

The convolutional neural network used for this task comprises the following components:

1.3.1 Convolutional Layers

The model contains multiple convolutional layers that extract hierarchical features from the input images. The operation is defined as:

$$Z_{m,n,k} = \sum_{i=1}^{H} \sum_{j=1}^{W} X_{m+i,n+j,c} W_{i,j,c,k} + b_k, \tag{2}$$

where $X_{m,n,c}$ represents the input at spatial location (m, n) with channel c, W is the convolutional filter of dimensions $H \times W \times C \times K$, and b_k is the bias term for the k-th filter. ReLU activation is applied:

$$\text{ReLU}(z) = \max(0, z). \tag{3}$$

The specific architecture consists of:

- A 16-filter convolutional layer with 3×3 kernels, followed by a max-pooling layer with 2×2 pooling size.
- A 32-filter convolutional layer with 3×3 kernels, followed by a max-pooling layer with 2×2 pooling size.
- A 64-filter convolutional layer with 3×3 kernels, followed by dropout regularization ($p = 0.5$) and a max-pooling layer.
- A 128-filter convolutional layer with 3×3 kernels, followed by a max-pooling layer.
- A 256-filter convolutional layer with 3×3 kernels, followed by a max-pooling layer.

1.3.2 Pooling Layers

Pooling layers reduce the spatial dimensions of the feature maps while retaining important information. Max pooling is defined as:

$$P_{m,n,k} = \max_{i=1}^{R} \max_{j=1}^{S} Z_{m+i,n+j,k}, \tag{4}$$

where $R \times S$ is the size of the pooling window.

1.3.3 Fully Connected Layers

The flattened feature maps are passed to fully connected layers. The architecture includes:

- A dense layer with 128 neurons and ReLU activation.
- A dropout layer ($p = 0.5$).
- A final dense layer with 3 neurons and softmax activation for class probabilities.

The softmax function is defined as:

$$\sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}, \tag{5}$$

where z_i is the input to the i-th class, and K is the total number of classes.

1.4 Model Algorithm

The training and evaluation process of the CNN model can be summarized using the following algorithm:

Algorithm 1 Training and Evaluation of CNN Model

Input: Training dataset $\mathcal{D}_{\text{train}}$, Validation dataset $\mathcal{D}_{\text{val}}$, Learning rate η, Number of epochs E
Output: Trained model parameters θ
Initialize model parameters θ
for $e = 1$ to E **do**
  Shuffle training dataset $\mathcal{D}_{\text{train}}$
  for each mini-batch B in $\mathcal{D}_{\text{train}}$ **do**
    Compute predictions $\hat{y}$ using forward pass
    Compute loss $\mathcal{L}$ using categorical cross-entropy:

$$\mathcal{L} = -\frac{1}{|B|} \sum_{i \in B} \sum_{j=1}^{K} y_{i,j} \log(\hat{y}_{i,j})$$

    Compute gradients $\nabla_\theta \mathcal{L}$
    Update parameters: $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$
  end for
  Evaluate model on $\mathcal{D}_{\text{val}}$
end for
Return optimized parameters θ =0

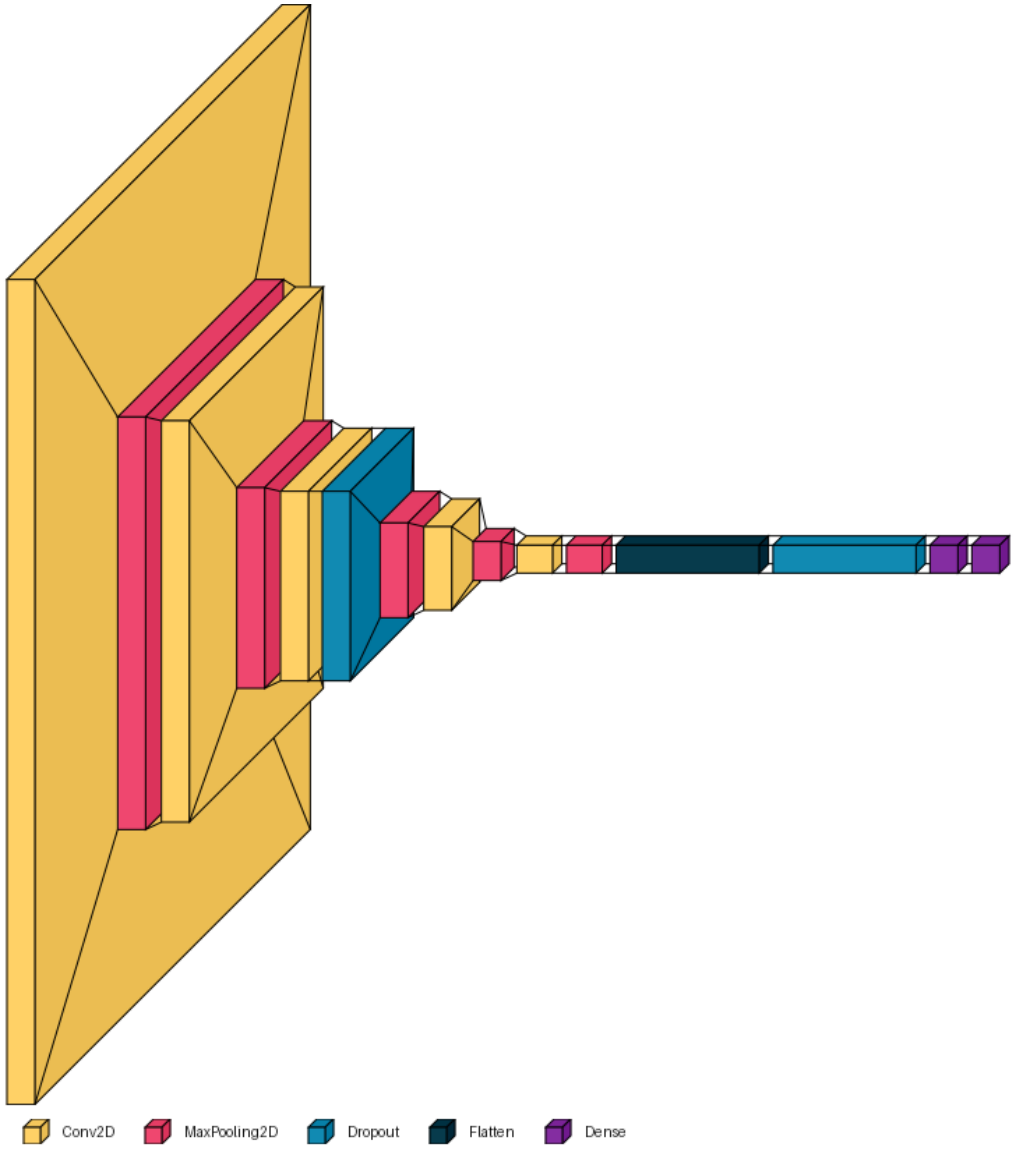


Figure 1: This picture shows the model architecture used for shoe classification.

1.5 Loss Function and Optimization

The model is optimized using the categorical cross-entropy loss function:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{K} y_{i,j}\log(\hat{y}_{i,j}), \tag{6}$$

where $y_{i,j}$ is the true label, $\hat{y}_{i,j}$ is the predicted probability for the j-th class, and N is the number of samples. Stochastic gradient descent (SGD) or its variants like Adam are used to minimize the loss function during training:

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}, \tag{7}$$

where θ represents the model parameters, η is the learning rate, and $\nabla_\theta \mathcal{L}$ is the gradient of the loss with respect to θ.

1.6 Evaluation Results

The evaluation of the model was conducted using the test dataset, and the results are summarized below. First, we reset the test data generator and generate predictions using the trained model. The model's output predictions were processed to classify them into the corresponding classes.

$$\text{predictions} = \text{model.predict}(\text{test\_generator})$$

$$\text{predicted\_classes} = \text{predictions.argmax}(axis = 1)$$

The true class labels for the test data were obtained as follows:

$$\text{true\_classes} = \text{test\_generator.classes}$$

The corresponding class labels are given by:

$$\text{class\_labels} = \text{list}(\text{test\_generator.class\_indices.keys}())$$

To assess the performance of the model, we computed several evaluation metrics. The classification report, which includes precision, recall, and F1-score for each class, was generated as follows:

$$\text{report} = \text{classification\_report}(\text{true\_classes}, \text{predicted\_classes}, \text{target\_names} = \text{class\_labels})$$

In addition to the classification report, we computed the macro-averaged precision, recall, and F1-score using the following formulas:

$$\text{Precision (Macro)} = \frac{1}{C} \sum_{i=1}^{C} \frac{TP_i}{TP_i + FP_i}$$

$$\text{Recall (Macro)} = \frac{1}{C} \sum_{i=1}^{C} \frac{TP_i}{TP_i + FN_i}$$

$$\text{F1 Score (Macro)} = \frac{2 \times \text{Precision (Macro)} \times \text{Recall (Macro)}}{\text{Precision (Macro)} + \text{Recall (Macro)}}$$

Where C is the number of classes, TP_i is the number of true positives, FP_i is the number of false positives, and FN_i is the number of false negatives for the i-th class.

The macro-averaged values are:

$$\text{Precision (Macro)} = \text{precision}$$

$$\text{Recall (Macro)} = \text{recall}$$

$$\text{F1 Score (Macro)} = \text{f1\_score}$$

Next, we also calculated the micro-averaged precision, recall, and F1-score, which are computed as follows:

$$\text{Precision (Micro)} = \frac{\sum_{i=1}^{C} TP_i}{\sum_{i=1}^{C} (TP_i + FP_i)}$$

$$\text{Recall (Micro)} = \frac{\sum_{i=1}^{C} TP_i}{\sum_{i=1}^{C} (TP_i + FN_i)}$$

$$\text{F1 Score (Micro)} = \frac{2 \times \text{Precision (Micro)} \times \text{Recall (Micro)}}{\text{Precision (Micro)} + \text{Recall (Micro)}}$$

The micro-averaged values are:

$$\text{Precision (Micro)} = \text{precision\_micro}$$

$$\text{Recall (Micro)} = \text{recall\_micro}$$

$$\text{F1 Score (Micro)} = \text{f1\_micro}$$

These metrics provide a comprehensive view of the model's classification performance across both individual classes and the entire dataset.

Class	Precision	Recall	F1-Score	Support
Adidas	0.50	0.71	0.59	38
Converse	0.68	0.45	0.54	38
Nike	0.49	0.45	0.47	38
Accuracy			0.54	114
Macro Avg	0.56	0.54	0.53	114
Weighted Avg	0.56	0.54	0.53	114

Table 1: Classification Report. The table summarizes the precision, recall, and F1-score for each class (Adidas, Converse, and Nike), as well as overall accuracy and average metrics. Adidas has the highest recall, while Converse has the best precision. The overall accuracy is 0.54, with macro and weighted averages showing slightly higher precision than recall.

Metric	Value
Precision (Macro)	0.56
Recall (Macro)	0.54
F1 Score (Macro)	0.53

Table 2: Macro Scores. The macro averages indicate that precision (0.56) is slightly higher than recall (0.54), with the overall F1 score (0.53) reflecting this imbalance. This suggests a relatively better performance in precision across all classes compared to recall.

Metric	Value
Precision (Micro)	0.54
Recall (Micro)	0.54
F1 Score (Micro)	0.54

Table 3: Micro Scores. The micro averages show equal performance for precision, recall, and F1 score (all 0.54). This indicates a balanced performance across the three classes when aggregated, meaning the model's overall classification accuracy is consistent for each individual metric.

1.7 Training and Validation Metrics Visualization

1.8 Training and Validation Metrics Visualization

In this section, I visualize the training and validation metrics of a machine learning model across epochs. Specifically, I focus on two key metrics: accuracy and loss. By tracking these metrics during the training process, I can evaluate how well the model is performing and whether it is overfitting or underfitting.

1.8.1 Setting the Visual Style

The first step involves setting the visual style for the plots. I apply a white grid style with a muted color palette and increase the font scale for better readability. This ensures that the visual output is clean, clear, and aesthetically pleasing, making it easier to interpret the trends. The visual style also helps highlight key patterns in the model's performance, facilitating an intuitive understanding of the training process.

1.8.2 Data Extraction and Formatting

Next, I extract the relevant data from the training history, which includes both training and validation accuracy, as well as training and validation loss. These values are organized into a structured format suitable for plotting. I store this data in a DataFrame to allow for easier manipulation and plotting. This structured format ensures that the metrics are aligned with the corresponding epochs, making it possible to visualize the model's progress across the training process.

1.8.3 Visualizing Accuracy

The first plot visualizes both the training and validation accuracy as the model progresses through each epoch. This allows me to observe how well the model is learning from the training data and how it generalizes to unseen validation data. Accuracy is a fundamental metric for classification tasks, providing insight into the proportion of correct predictions made by the model.

If the model shows a significant gap between training and validation accuracy, it may indicate overfitting. Overfitting occurs when the model performs well on the training data but fails to generalize to new, unseen data (i.e., the validation set). In such cases, the model may be memorizing specific patterns in the training data rather than learning generalized patterns that apply to unseen data.

1.8.4 Visualizing Loss

The second plot shows the training and validation loss across epochs. Loss is another important metric, as it reflects the error made by the model during training. Lower loss values indicate that the model is making fewer errors, which is desirable. In an ideal scenario, both training and validation losses should decrease over time, reflecting that the model is improving and fitting both the training and validation datasets well.

If the validation loss begins to increase while the training loss continues to decrease, it could indicate overfitting. In this case, the model is likely becoming too specialized to the training data and failing to generalize well to unseen data. Monitoring both accuracy and loss helps in understanding whether the model is truly learning and generalizing, or whether it needs adjustments to avoid overfitting.

The second plot shows the training and validation loss across epochs. Loss is a key indicator of how well the model is minimizing error during training. In an ideal scenario, both training and validation loss should decrease and converge as the model improves. If the validation loss starts to increase while the training loss continues to decrease, it may indicate overfitting, where the model is becoming too specialized to the training data and unable to generalize to the validation set.

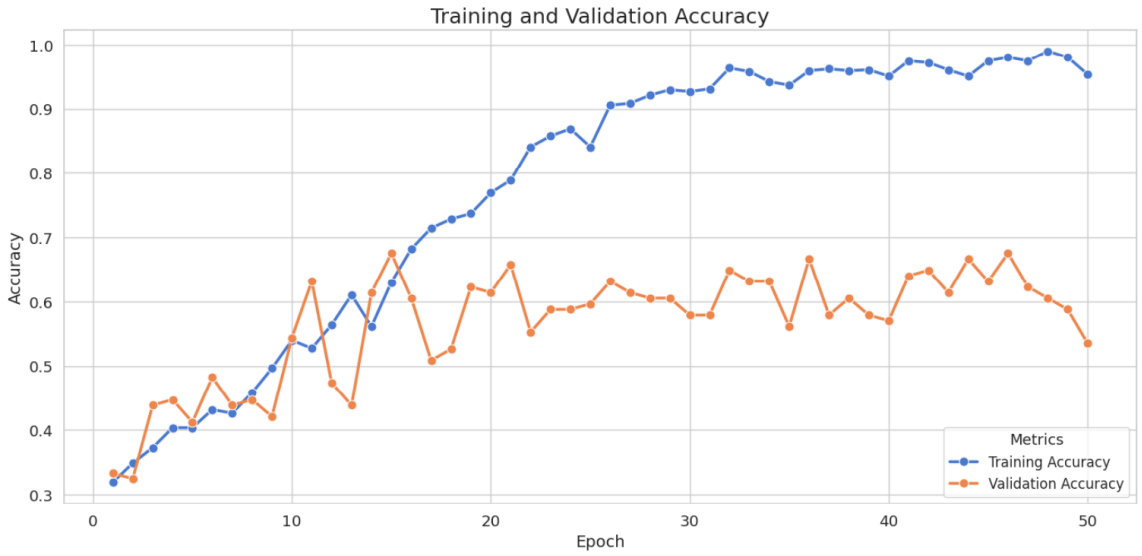


Figure 2: Training and Validation Accuracy across Epochs. This plot shows the accuracy for both the training and validation datasets over the course of training. The curves help us observe trends such as overfitting, where validation accuracy may plateau or decrease as training accuracy increases.

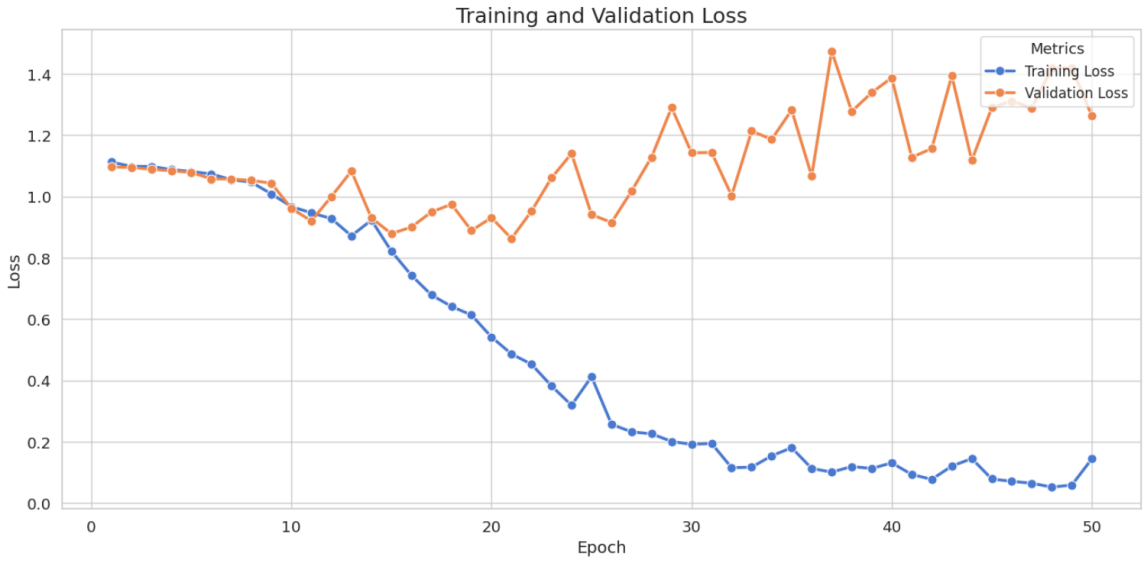


Figure 3: Training and Validation Loss across Epochs. This plot shows the loss for both the training and validation datasets. Ideally, both the training and validation loss should decrease and converge, indicating that the model is learning and generalizing well.

1.9 Hyperparameter Testing Explanation

In this section, I will explain the key hyperparameters used in the model, their mathematical interpretation, and how they influence the training process. The three hyperparameters that are independently tested in this code are **filters**, **dropout rate**, and **learning rate**.

1.9.1 Filters in Convolutional Layers

The number of **filters** in the convolutional layers determines the number of feature maps that will be created from the input images. These filters act as feature detectors, learning various patterns (such as edges, textures, or shapes) during training.

Mathematically, the number of filters F affects the output shape of the convolution operation. For an input image of size $H \times W \times C$ (height, width, channels), and a filter size of $f \times f$, the output dimension of the convolutional layer is given by:

$$\text{Output Height} = \left\lfloor \frac{H - f}{S} + 1 \right\rfloor$$

$$\text{Output Width} = \left\lfloor \frac{W - f}{S} + 1 \right\rfloor$$

where S is the stride. Increasing the number of filters captures more complex patterns, which can lead to a better understanding of the data, but it also increases the model's computational cost.

In the code, the `filters` hyperparameter is tested with different values: $(16, 32, 64, 128, 256)$, $(32, 64, 128, 256, 512)$, and $(64, 128, 256, 512, 1024)$, to determine the optimal number of feature maps required to extract meaningful features from the data.

1.9.2 Dropout Rate

The **dropout rate** is a regularization technique used to prevent overfitting by randomly setting a fraction of input units to zero during training. This forces the network to learn redundant representations, thus improving its ability to generalize.

Mathematically, during training, at each step, a fraction p of the inputs is dropped (set to zero), and the remaining inputs are scaled by $\frac{1}{1-p}$. This can be represented as:

$$\hat{x} = \frac{x}{1-p} \quad \text{for} \quad \text{input } x \text{ with probability } p \text{ of being dropped.}$$

The dropout rate is tested with different values: 0.3, 0.5, and 0.7 in the code. A higher dropout rate might prevent overfitting by ensuring that the model does not overly rely on any particular feature, while a very high rate can lead to underfitting as the model may not have enough information to learn effectively.

1.9.3 Learning Rate

The **learning rate** controls the step size during the optimization process. It determines how much the model weights are updated with respect to the gradient of the loss function.

Mathematically, the update rule for the weights W in an optimizer like Adam is given by:

$$W_{t+1} = W_t - \eta \cdot \nabla L(W_t)$$

where η is the learning rate, $\nabla L(W_t)$ is the gradient of the loss L at time step t, and W_t is the weight at time step t.

The learning rate is tested with values of 0.0001, 0.001, and 0.01 in the code. A small learning rate may result in slow convergence, while a large learning rate may cause the model to converge too quickly or even diverge.

1.9.4 Summary

In summary, the three hyperparameters tested in this model play crucial roles in its performance. The filters affect the model's capacity to learn features from the data, the dropout rate helps prevent overfitting by regularizing the training process, and the learning rate controls the speed at which the model learns. Tuning these hyperparameters helps achieve a balance between underfitting and overfitting, leading to improved generalization on unseen data.

1.10 Results and Analysis

In this section, we present the results of training our model with different hyperparameter configurations. The accuracy and loss values are reported for both training and validation datasets across 12 epochs.

1.10.1 Results Tables

Filters: (32, 64, 128, 256, 512)

Epoch	Training Accuracy	Training Loss	Validation Accuracy	Validation Loss
1	0.3161	1.1195	0.3333	1.0974
2	0.3372	1.0990	0.3333	1.0975
3	0.3612	1.0978	0.3333	1.0986
12	0.3977	1.0822	0.4649	1.0394

Table 4: Performance with filters (32, 64, 128, 256, 512).

Filters: (64, 128, 256, 512, 1024)

Epoch	Training Accuracy	Training Loss	Validation Accuracy	Validation Loss
1	0.3266	1.3404	0.3333	1.1013
2	0.3364	1.0965	0.3333	1.0972
3	0.3491	1.0998	0.3333	1.0988
12	0.3658	1.0932	0.3333	1.1001

Table 5: Performance with filters (64, 128, 256, 512, 1024).

Dropout Rate: 0.3

Epoch	Training Accuracy	Training Loss	Validation Accuracy	Validation Loss
1	0.2929	1.1090	0.3333	1.0936
2	0.3645	1.0963	0.4123	1.0931
3	0.3987	1.0934	0.4649	1.0666
12	0.4696	1.0400	0.5877	0.9323

Table 6: Performance with dropout rate 0.3.

Dropout Rate: 0.5

Epoch	Training Accuracy	Training Loss	Validation Accuracy	Validation Loss
1	0.3119	1.1375	0.3421	1.0973
2	0.3282	1.0986	0.3246	1.0927
3	0.3163	1.1029	0.4035	1.0972
12	0.3658	1.0932	0.3333	1.1001

Table 7: Performance with dropout rate 0.5.

1.10.2 Analysis

Filters: (32, 64, 128, 256, 512) The model showed a gradual improvement in training accuracy and reduction in training loss, reaching 39.77% accuracy by the 12th epoch. Validation accuracy also improved to 46.49%. The reduction in validation loss indicates effective learning.

Filters: (64, 128, 256, 512, 1024) The deeper filter configuration exhibited slower improvement. Training and validation accuracy plateaued at approximately 36%. The high validation loss suggests overfitting or insufficient learning.

Dropout Rate: 0.3 Incorporating a moderate dropout rate led to a significant improvement, achieving a validation accuracy of 58.77% in the 12th epoch. The decrease in validation loss to 0.9323 highlights better generalization.

Dropout Rate: 0.5 A higher dropout rate did not yield substantial benefits, with validation accuracy peaking at 40.35%. Validation loss remained consistently high, suggesting underfitting due to excessive regularization.

1.11 Classification Reports

Configuration: Filters = (16, 32, 64, 128, 256), Dropout Rate = 0.5, Learning Rate = 0.001

Class	Precision	Recall	F1-Score
Adidas	0.45	0.50	0.47
Converse	0.36	0.68	0.47
Nike	0.00	0.00	0.00
Accuracy		0.39	
Macro Avg	0.27	0.39	0.32
Weighted Avg	0.27	0.39	0.32

Configuration: Filters = (32, 64, 128, 256, 512), Dropout Rate = 0.5, Learning Rate = 0.001

Class	Precision	Recall	F1-Score
Adidas	0.32	0.58	0.41
Converse	0.31	0.13	0.19
Nike	0.34	0.26	0.30
Accuracy		0.32	
Macro Avg	0.33	0.32	0.30
Weighted Avg	0.33	0.32	0.30

Configuration: Filters = (64, 128, 256, 512, 1024), Dropout Rate = 0.5, Learning Rate = 0.001

Class	Precision	Recall	F1-Score
Adidas	0.00	0.00	0.00
Converse	0.33	1.00	0.50
Nike	0.00	0.00	0.00
Accuracy		0.33	
Macro Avg	0.11	0.33	0.17
Weighted Avg	0.11	0.33	0.17

Configuration: Filters = (16, 32, 64, 128, 256), Dropout Rate = 0.3, Learning Rate = 0.001 Configuration: Filters = (16, 32, 64, 128, 256), Dropout Rate = 0.7,

Class	Precision	Recall	F1-Score
Adidas	0.33	0.42	0.37
Converse	0.33	0.39	0.36
Nike	0.20	0.11	0.14
Accuracy		0.31	
Macro Avg	0.29	0.31	0.29
Weighted Avg	0.29	0.31	0.29

Learning Rate = 0.001

Class	Precision	Recall	F1-Score
Adidas	0.24	0.32	0.27
Converse	0.25	0.08	0.12
Nike	0.29	0.39	0.33
Accuracy		0.26	
Macro Avg	0.26	0.26	0.24
Weighted Avg	0.26	0.26	0.24

Configuration: Filters = (16, 32, 64, 128, 256), Dropout Rate = 0.5, Learning Rate = 0.0001

Class	Precision	Recall	F1-Score
Adidas	0.31	0.34	0.33
Converse	0.00	0.00	0.00
Nike	0.30	0.55	0.39
Accuracy		0.30	
Macro Avg	0.20	0.30	0.24
Weighted Avg	0.20	0.30	0.24

Configuration: Filters = (16, 32, 64, 128, 256), Dropout Rate = 0.5, Learning Rate = 0.01

Class	Precision	Recall	F1-Score
Adidas	0.00	0.00	0.00
Converse	0.33	1.00	0.50
Nike	0.00	0.00	0.00
Accuracy		0.33	
Macro Avg	0.11	0.33	0.17
Weighted Avg	0.11	0.33	0.17

2 Q1.Convolution Neural Network for Image Classification (Extra Point)

2.1 Overall Structure

The architecture consists of several key components:

- Initial convolutional layers for feature extraction.
- Residual blocks with multiple convolutional layers.
- Batch normalization for stabilizing training.
- A final global average pooling layer to reduce the spatial dimensions.
- Fully connected layers for classification.

2.2 Initial Convolutional Layer

The input image is passed through an initial convolutional layer with a kernel size of 7×7, a stride of 2, and padding to ensure the output dimensions are consistent. This layer is followed by batch normalization and a ReLU activation function.

The mathematical operation for this convolution is as follows:

$$\mathbf{y}(i,j) = \sum_{m} \sum_{n} \mathbf{f}(m,n) \cdot \mathbf{x}(i+m, j+n)$$

where:

- $\mathbf{x}$ is the input image,
- $\mathbf{f}$ is the filter or kernel,
- $\mathbf{y}$ is the output feature map.

2.3 Residual Blocks

After the initial layer, the network consists of multiple residual blocks. Each residual block is designed to learn a residual mapping instead of the direct mapping. The residual block consists of two or three convolutional layers, each followed by batch normalization and ReLU activations.

A residual block can be represented mathematically as:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \mathbf{W}) + \mathbf{x}$$

where:

- $\mathcal{F}(\mathbf{x}, \mathbf{W})$ is the transformation applied to the input $\mathbf{x}$,
- $\mathbf{W}$ represents the weights of the convolutional layers in the block,
- The input $\mathbf{x}$ is added directly to the output of the transformation, forming the residual connection.

This design allows the network to train more efficiently and helps mitigate the vanishing gradient problem by ensuring gradients can propagate directly through the residual connections.

Each residual block is followed by a shortcut connection, which skips the block's layers and directly adds the input to the output. This shortcut connection helps preserve the original input information and ensures more effective training.

2.4 Bottleneck Design

In some of the residual blocks, a bottleneck design is used to reduce the number of parameters and computational complexity. This design includes three layers:

- A 1×1 convolution to reduce the input dimensions,
- A 3×3 convolution to extract features,
- A 1×1 convolution to increase the output dimensions.

The bottleneck design reduces the computational cost while maintaining the ability to learn complex features.

2.5 Global Average Pooling Layer

At the end of the residual blocks, a global average pooling layer is used to reduce the spatial dimensions of the feature maps to a single value per channel. This operation is mathematically defined as:

$$\mathbf{y}_i = \frac{1}{H \times W} \sum_{h=1}^{H} \sum_{w=1}^{W} \mathbf{x}_{h,w,i}$$

where:

- $\mathbf{x}_{h,w,i}$ is the feature map at position (h, w) in channel i,
- H and W are the height and width of the feature map.

The output is a vector of size equal to the number of channels, which is then passed to a fully connected layer for classification.

2.6 Fully Connected Layer

The output from the global average pooling layer is passed through a fully connected (dense) layer, which performs the following matrix multiplication:

$$\mathbf{y} = \mathbf{W} \cdot \mathbf{x} + \mathbf{b}$$

where:

- $\mathbf{W}$ is the weight matrix of the fully connected layer,
- $\mathbf{x}$ is the output of the pooling layer,
- $\mathbf{b}$ is the bias term,
- $\mathbf{y}$ is the final output vector.

The final layer applies a softmax activation function to produce class probabilities:

$$\mathbf{p}_i = \frac{\exp(\mathbf{y}_i)}{\sum_{j=1}^{C} \exp(\mathbf{y}_j)}$$

where C is the number of classes, and $\mathbf{p}_i$ is the predicted probability for class i.

2.7 Feature Extraction: Model Backbone

The pretrained ResNet-50 is utilized as the **backbone** of the model, serving as the feature extractor. This backbone processes the input image $\mathbf{x}$ through a sequence of convolutional operations, denoted as f_{conv}, to produce a high-dimensional feature vector $\mathbf{F}$:

$$\mathbf{F} = f_{\text{conv}}(\mathbf{x}), \quad \mathbf{F} \in \mathbb{R}^{2048}$$

In this architecture, the fully connected (fc) layer of ResNet-50 is replaced with an identity mapping. This ensures that the output is the feature vector $\mathbf{F}$, which is then passed to the classification head for further processing.

2.8 Classification Head

The classification head consists of several fully connected layers, ReLU activations, and dropout layers. The head maps the feature vector $\mathbf{F}$ to the final output:

$$\mathbf{y} = f_{\text{head}}(\mathbf{F})$$

where f_{head} is a composition of transformations applied sequentially:

$$f_{\text{head}}(\mathbf{F}) = f_6 \circ f_5 \circ f_4 \circ f_3 \circ f_2 \circ f_1(\mathbf{F})$$

Each layer is defined as:

$$f_i(\mathbf{z}) = \text{ReLU}(\mathbf{W}_i\mathbf{z} + \mathbf{b}_i) \quad \text{for } i = 1, 2, \ldots, 5$$

where:

- $\mathbf{W}_i$ and $\mathbf{b}_i$ are the weights and biases of the i-th linear layer,
- $\text{ReLU}(x) = \max(0, x)$ is the Rectified Linear Unit activation function,
- Dropout is applied after each activation to prevent overfitting.

The sizes of the transformations are as follows:

$$\begin{aligned}
f_1 &: \mathbb{R}^{2048} \to \mathbb{R}^{1024} \\
f_2 &: \mathbb{R}^{1024} \to \mathbb{R}^{512} \\
f_3 &: \mathbb{R}^{512} \to \mathbb{R}^{256} \\
f_4 &: \mathbb{R}^{256} \to \mathbb{R}^{128} \\
f_5 &: \mathbb{R}^{128} \to \mathbb{R}^{64} \\
f_6 &: \mathbb{R}^{64} \to \mathbb{R}^{\text{num_classes}}
\end{aligned}$$

2.9 Results and Analysis

2.9.1 Classification Report

The model's performance was evaluated using precision, recall, and F1-score metrics for each class, along with macro and micro averages. The results are presented in the table below:

Class	Precision	Recall	F1-Score	Support
`adidas`	0.85	0.89	0.87	37
`converse`	0.91	0.89	0.90	55
`nike`	0.92	0.90	0.91	50
Accuracy			**0.89**	142
Macro Avg	0.89	0.89	0.89	142
Weighted Avg	0.90	0.89	0.89	142

Table 8: Classification report showing precision, recall, F1-score, and support for each class.

2.9.2 Analysis of Precision, Recall, and F1-Score

Precision, **Recall**, and **F1-score** are key metrics for evaluating classification models. These metrics provide insight into how well the model distinguishes between classes, correctly identifies relevant samples, and balances false positives and negatives. The following observations can be made from the table above:

- The precision for `adidas` is 0.85, meaning 85% of the samples predicted as `adidas` were correctly classified. The model is less precise for `adidas` compared to `nike` and `converse`.
- The precision for `converse` is 0.91, reflecting a high accuracy in predicting this class.
- The precision for `nike` is the highest at 0.92, suggesting that the model is very accurate when predicting `nike`.
- The recall for `adidas` is 0.89, meaning that 89% of actual `adidas` samples were correctly identified by the model. This indicates that the model is fairly good at recognizing `adidas`, but there may still be some confusion with other classes.

- The recall for `converse` is 0.89, showing that the model captures most of the `converse` samples correctly.
- The recall for `nike` is 0.90, suggesting that the model has a high ability to recognize `nike` samples.
- The F1-scores provide a balance between precision and recall. For `adidas`, the F1-score is 0.87, indicating a good trade-off between precision and recall. The F1-score for `converse` is 0.90, and for `nike`, it is 0.91, which are the highest among the classes, indicating very balanced performance.

2.9.3 Macro and Micro Averages

The **Macro Average** and **Micro Average** are important metrics for evaluating overall model performance across all classes.

- The **Macro Average** is computed by averaging the precision, recall, and F1-score for all classes. In this case, the macro averages are:

  $$\text{Precision (Macro)} = 0.89, \quad \text{Recall (Macro)} = 0.89, \quad \text{F1-Score (Macro)} = 0.89$$

  These values indicate that the model performs consistently well across all three classes without favoring any particular class.

- The **Micro Average** calculates metrics globally by counting the total true positives, false positives, and false negatives across all classes. The micro averages for precision, recall, and F1-score are:

  $$\text{Precision (Micro)} = 0.89, \quad \text{Recall (Micro)} = 0.89, \quad \text{F1-Score (Micro)} = 0.89$$

  This further reinforces the idea that the model performs well globally, without bias towards any specific class.

2.9.4 Conclusion

The model demonstrates solid performance across all three classes, achieving an overall accuracy of 0.89. Specific observations include:

- The `nike` class has the highest precision and recall, suggesting that the model excels at identifying this category.

- The `adidas` class has slightly lower performance metrics, particularly in terms of precision, indicating that there may be some room for improvement in distinguishing `adidas` from other classes.
- The macro and micro averages are both 0.89, confirming that the model performs consistently well across all classes.

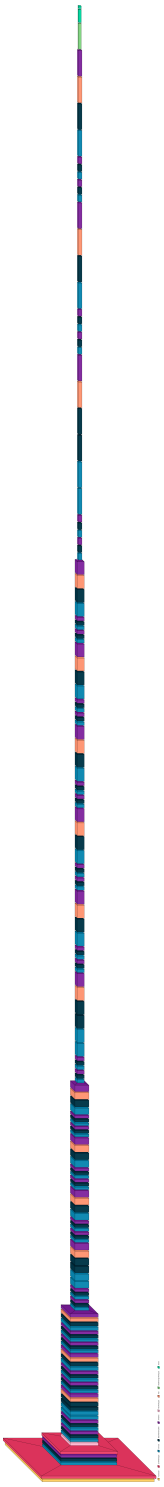


Figure 4: Architecture of the model used for classification tasks. The model architecture consists of several layers, including convolutional layers, pooling layers, and fully connected layers, which work together to extract features from input images and classify them into categories such as `adidas`, `converse`, and `nike`.

3 Q2.Convolution Neural Network And U-Net for Image Denoising

3.1 Introduction

In this report, we describe a model for image denoising based on the U-Net architecture. The model is designed to remove noise from images while preserving important features. We discuss the architecture in detail and evaluate its performance using several image quality metrics.

3.2 Data Preprocessing

The dataset used in this project is the FER-2013 dataset, which consists of images with corresponding pixel values. The images are processed into a square format and normalized to values between 0 and 1. A Gaussian noise is then added to the images to simulate noisy inputs.

3.3 Model Architecture

The model is based on the U-Net architecture, which is composed of an encoder-decoder structure. Below is the step-by-step description of the model:

3.3.1 Encoder

The encoder consists of several blocks, each containing:

- Two convolutional layers with ReLU activations.
- Batch normalization after each convolution.
- Max-pooling layer to reduce spatial dimensions.

Each convolutional block in the encoder is followed by a max-pooling operation, which reduces the size of the feature map by a factor of 2, effectively down-sampling the image.

3.3.2 Bottleneck

The bottleneck of the U-Net architecture consists of two convolutional layers with 1024 filters, followed by batch normalization.

3.3.3 Decoder

The decoder consists of several blocks, each containing:

- A transposed convolutional layer to up-sample the feature maps.
- Concatenation with the corresponding feature map from the encoder (skip connections).
- Two convolutional layers with ReLU activations and batch normalization.

The output of the decoder is passed through a final convolutional layer with a sigmoid activation to produce the denoised image.

3.3.4 Mathematical Representation

Let the noisy input image be denoted by $\mathbf{x}_{noisy}$ and the output denoised image by $\mathbf{x}_{denoised}$. The U-Net architecture can be mathematically represented as:

$$\mathbf{x}_{denoised} = \text{U-Net}(\mathbf{x}_{noisy})$$

Where the U-Net function involves the sequence of convolutions, activations, max-pooling, and up-sampling operations discussed above.

3.4 Metrics

The model is evaluated using several image quality metrics:

3.4.1 Peak Signal-to-Noise Ratio (PSNR)

PSNR measures the ratio between the maximum possible power of a signal and the power of distorting noise. It is defined as:

$$\text{PSNR}(X, Y) = 10 \cdot \log_{10} \left(\frac{MAX_I^2}{MSE(X, Y)} \right)$$

Where:

$$MSE(X, Y) = \frac{1}{N} \sum_{i=1}^{N} (X_i - Y_i)^2$$

Here, X and Y are the original and denoised images, respectively, and MAX_I is the maximum pixel value (255 for an 8-bit image).

3.4.2 Structural Similarity Index (SSIM)

SSIM evaluates the perceptual quality of images by considering luminance, contrast, and structure. It is defined as:

$$\text{SSIM}(X, Y) = \frac{(2\mu_X\mu_Y + c_1)(2\sigma_{XY} + c_2)}{(\mu_X^2 + \mu_Y^2 + c_1)(\sigma_X^2 + \sigma_Y^2 + c_2)}$$

Where μ_X and μ_Y are the means of X and Y, σ_X^2 and σ_Y^2 are the variances, and σ_{XY} is the covariance between X and Y. Constants c_1 and c_2 are used to stabilize the division.

3.4.3 Edge Preservation Index (EPI)

EPI measures the preservation of edges in the denoised image. It is defined as:

$$EPI = \frac{\sum \text{edges}_{denoised}}{\max(\sum \text{edges}_{noisy}, 1)}$$

Where edges refers to the edges detected in the image using the Canny edge detector.

3.4.4 Image Gradient Similarity (IGS)

IGS measures the similarity in the gradient between the noisy and denoised images:

$$IGS = \frac{1}{N} \sum_{i=1}^{N} |\nabla X_i - \nabla Y_i|$$

Where ∇X and ∇Y represent the gradients of the noisy and denoised images, respectively.

3.5 Algorithm

The model training process can be described in the following algorithm:

Algorithm 2 Training the Denoising Model

Input: Noisy images X_{noisy}, original images X_{clean}
Output: Trained model model
Preprocess images to fit the input shape:
  $X_{\text{preprocessed}} = \text{Preprocess}(X_{\text{noisy}}, X_{\text{clean}})$
Split the dataset into training and testing sets:
  $(X_{\text{train\_noisy}}, X_{\text{train\_clean}}), (X_{\text{test\_noisy}}, X_{\text{test\_clean}}) = \text{TrainTestSplit}(X_{\text{preprocessed}})$
Add Gaussian noise to the images to simulate noisy inputs:
  $X_{\text{noisy}} = X_{\text{clean}} + \mathcal{N}(0, \sigma^2)$
Build the U-Net denoising model:
  model = U-Net(input shape)
Compile the model with Adam optimizer and mean squared error loss:
  model.compile(optimizer = Adam, loss = MSE)
for epoch = 1 to number of epochs **do**
  Train the model on $X_{\text{train\_noisy}}$ and $X_{\text{train\_clean}}$:
    model.fit($X_{\text{train\_noisy}}, X_{\text{train\_clean}}$)
  Evaluate the model on the test set $X_{\text{test\_noisy}}$ and $X_{\text{test\_clean}}$:
    model.evaluate($X_{\text{test\_noisy}}, X_{\text{test\_clean}}$)
  Calculate metrics:
  PSNR (Peak Signal-to-Noise Ratio):
    $\text{PSNR} = 10 \cdot \log_{10}\left(\frac{I_{\max}^2}{\text{MSE}}\right)$
    where $I_{\max}$ is the maximum pixel value, and MSE is the Mean Squared Error:
    $\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}\left(X_{\text{predicted}}(i) - X_{\text{clean}}(i)\right)^2$
  SSIM (Structural Similarity Index):
    $\text{SSIM}(X_{\text{clean}}, X_{\text{predicted}}) = \frac{(2\mu_X\mu_Y + C_1)(2\sigma_{XY} + C_2)}{(\mu_X^2 + \mu_Y^2 + C_1)(\sigma_X^2 + \sigma_Y^2 + C_2)}$
    where:
    μ_X, μ_Y are the means of X_{clean} and $X_{\text{predicted}}$,
    σ_X^2, σ_Y^2 are the variances of X_{clean} and $X_{\text{predicted}}$,
    σ_{XY} is the covariance between X_{clean} and $X_{\text{predicted}}$,
    C_1 and C_2 are constants to stabilize the division.
  EPI (Edge Preservation Index):
    $\text{EPI}(X_{\text{noisy}}, X_{\text{denoised}}) = \frac{\sum \text{Canny}(X_{\text{denoised}})}{\max(\sum \text{Canny}(X_{\text{noisy}}), 1)}$
    where $\text{Canny}(X)$ is the number of edges detected in the image X using the Canny edge detector.
  IGS (Image Gradient Similarity):
    $\text{IGS}(X_{\text{noisy}}, X_{\text{denoised}}) = \frac{1}{N}\sum_{i=1}^{N}|\nabla X_{\text{noisy}}(i) - \nabla X_{\text{denoised}}(i)|$
    where ∇X represents the gradient (edge information) of the image X, and N is the number of pixels.
end for
Return the trained model =0

3.5.1 Model Architecture

The architecture of the U-Net-CNN denoising model is shown in Figure 5. This model consists of several convolutional layers followed by transposed convolutional layers, with skip connections between the corresponding layers in the encoding and decoding parts.

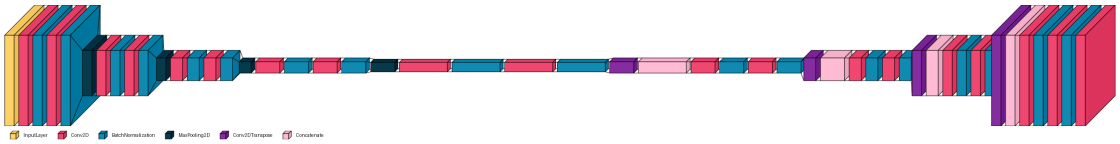


Figure 5: Architecture of the U-Net-CNN denoising model. The model follows an encoder-decoder structure with skip connections to preserve spatial information.

3.6 Conclusion

The U-Net architecture is effective for image denoising tasks, and the model achieves satisfactory performance as measured by PSNR, SSIM, EPI, and IGS.

3.7 Results

The performance of the denoising model was evaluated under different noise factors. We analyzed the results based on four metrics: Peak Signal-to-Noise Ratio (PSNR), Structural Similarity Index (SSIM), Edge Preservation Index (EPI), and Image Gradient Similarity (IGS). The results for various noise factors are summarized in the table below:

Noise Factor	PSNR (dB)	SSIM	EPI	IGS
0.2	22.8557	0.6562	0.5727	0.1017
0.1	26.9047	0.7917	0.7048	0.0492
0.05	29.5461	0.8571	0.8506	0.0248

Table 9: Denoising performance at various noise factors. The table shows the average PSNR, SSIM, EPI, and IGS values after applying the denoising model.

Noise Factor 0.2: At a higher noise factor, the model achieves a PSNR of 22.8557, which indicates a relatively lower quality of denoising compared to lower noise levels. The SSIM value of 0.6562 suggests moderate structural similarity between the denoised and original images. The EPI value of 0.5727 indicates reasonable

edge preservation, and the IGS value of 0.1017 suggests that the gradients in the noisy and denoised images are still somewhat dissimilar.

Noise Factor 0.05: At a lower noise factor, the denoising model performs significantly better. The PSNR increases to 29.5461, indicating much higher quality after denoising. The SSIM value of 0.8571 suggests that the model was able to preserve most of the structural details in the image. Both the EPI (0.8506) and IGS (0.0248) values are also improved, showing that the model effectively preserves edges and gradient details in the denoised images.

Noise Factor 0.1: For this intermediate noise factor, the performance is better than the high noise level but not as good as the low noise factor. The PSNR of 26.9047, SSIM of 0.7917, and EPI of 0.7048 suggest a balanced performance with decent edge and gradient preservation. The IGS value of 0.0492 shows that there is still some difference in the gradients between the noisy and denoised images.

Overall Conclusion: The results show that the denoising model performs best at lower noise factors, with a noticeable improvement in all metrics as the noise level is reduced.

3.7.1 Noisy and Denoised Images (Noise Level 0.05)

Figure 6 shows the image with 0.05 noise level and the denoised version of the image. The noise introduced slight disturbances in the image, but the denoising algorithm successfully restored its quality.

(a) Noisy Image (Noise Level 0.05)

Figure 6: Noisy and Denoised Images for Noise Level 0.05. The denoising algorithm significantly reduces the noise while maintaining image details.

3.7.2 Noisy and Denoised Images (Noise Level 0.2)

Figure shows the image with 0.2 noise level and its denoised version. The higher noise level resulted in more visible distortions, but the denoising process still improved the image quality.

(a) Noisy Image (Noise Level 0.2)

Figure 7: Noisy and Denoised Images for Noise Level 0.2. The denoising algorithm successfully reduces noise despite the higher noise level.

3.8 Training Metrics Visualization

The following sections describe the training metrics visualized during the training process of the model. The plots provide insights into the model's performance with respect to various metrics over the course of 30 epochs.

3.8.1 Average PSNR Over Epochs

Figure shows the average PSNR (Peak Signal-to-Noise Ratio) value across 30 epochs. PSNR is a key indicator of image quality, where higher values represent better quality. As the number of epochs increases, the PSNR value shows an improving trend, indicating better quality of the reconstructed images.

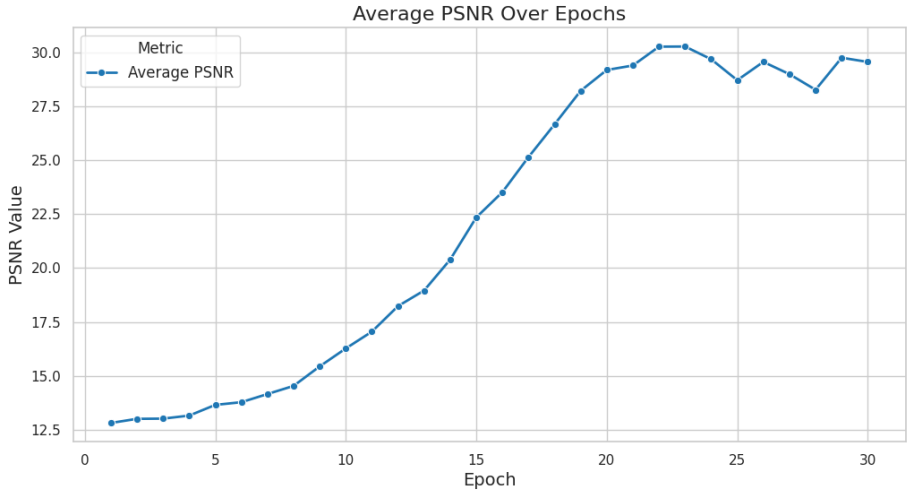


Figure 8: Average PSNR Over Epochs. The PSNR increases steadily, suggesting that the model improves image quality over time.

3.8.2 Training Metrics (SSIM, EPI, IGS) Over Epochs

Figure shows the trends of SSIM (Structural Similarity Index), EPI (Edge Preservation Index), and IGS (Image Gradient Similarity) over the 30 epochs. These metrics are essential for evaluating image quality in terms of structural similarity, edge preservation, and gradient consistency, respectively. As seen in the plot, SSIM and EPI show improvements, while IGS remains relatively stable.

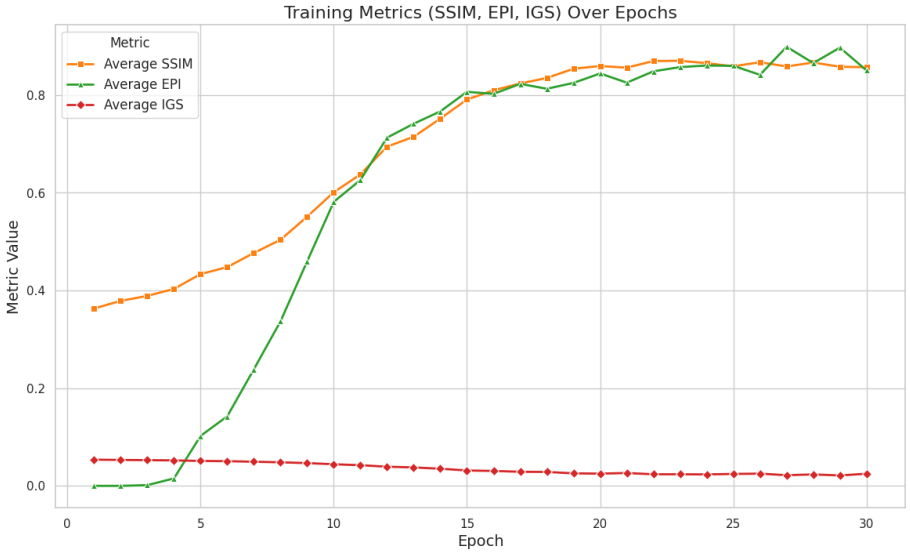


Figure 9: Training Metrics (SSIM, EPI, IGS) Over Epochs. The improvement in SSIM and EPI indicates that the model is learning to preserve the structure and edges of the image.

3.8.3 Loss and Validation Loss Over Epochs

Figure shows the loss and validation loss values over 30 epochs. Loss measures the difference between the predicted and actual values, while validation loss helps assess the model's performance on unseen data. The dual-axis plot clearly illustrates the reduction in both training and validation loss, indicating that the model is improving and generalizing well.

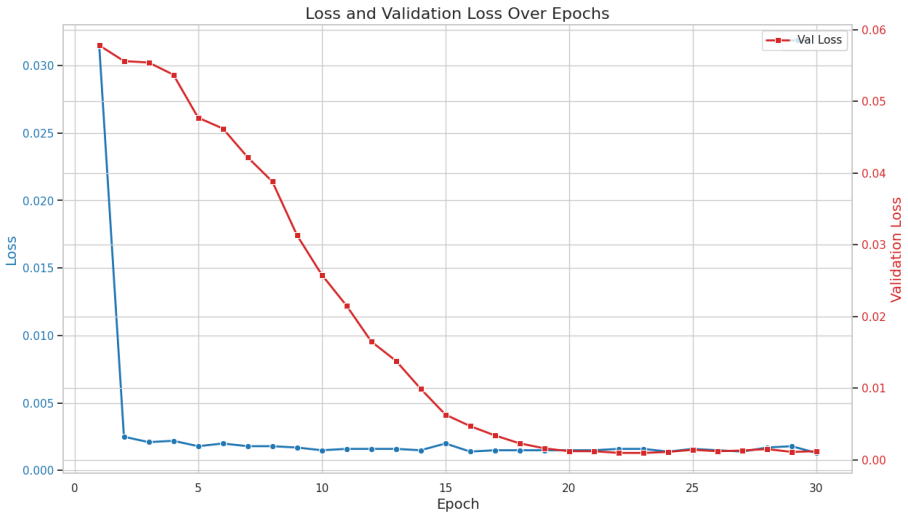


Figure 10: Loss and Validation Loss Over Epochs. Both the training and validation loss decrease, suggesting effective model training and generalization.

4 Q3. Region-Based Convolution Neural Network

4.1 Introduction

Object detection is a task in computer vision that involves detecting and localizing objects within an image. For this project, the objective is to detect airplanes in images, which is a crucial task in surveillance, satellite imagery, and other applications. The algorithm presented in this report leverages a deep learning approach based on convolutional neural networks (CNNs) to perform both classification and localization of objects.

4.2 Dataset

The dataset used for this task consists of images and their corresponding annotations. Each image is labeled with bounding boxes that mark the locations of airplanes. The annotations are stored in text files, where each line represents a bounding box with the following format:

$$\text{label} \quad x_{\min} \quad y_{\min} \quad x_{\max} \quad y_{\max}$$

where $x_{\min}, y_{\min}$ and $x_{\max}, y_{\max}$ are the coordinates of the top-left and bottom-right corners of the bounding box, respectively.

4.3 Dataset Split

The dataset is divided into two subsets: training and validation. The training set comprises 80% of the data, while the validation set consists of the remaining 20%. This split is used to train the model and evaluate its performance.

4.4 Model Architecture: Region-Based RCNN

The Region-Based Convolutional Neural Network (RCNN) is a model used for object detection, where the goal is to localize and classify objects within an image. The architecture consists of several stages, including region proposal, feature extraction, and classification.

4.4.1 Overview

The architecture consists of two main components: 1. **Region Proposal Network (Selective Search)**: This generates potential object regions (proposals) from an image. 2. **Region-Based CNN (RCNN)**: This component extracts features from the proposed regions and classifies them.

4.4.2 Region Proposal: Selective Search

Given an input image I, we first generate region proposals using the selective search algorithm. The image I is divided into superpixels, and regions are merged based on similarity metrics. Let $R_1, R_2, \ldots, R_k$ denote the proposed regions (bounding boxes) generated by the algorithm.

Each region R_i is defined as:

$$R_i = \{(x_{\min}, y_{\min}), (x_{\max}, y_{\max})\}$$

where $(x_{\min}, y_{\min})$ and $(x_{\max}, y_{\max})$ represent the top-left and bottom-right coordinates of the bounding box for the i-th region.

4.4.3 Feature Extraction and Classification

Once the region proposals are generated, the next step is to extract features from each region. For each proposed region R_i, we extract a fixed-size region from the input image I and resize it to a size of 224×224 pixels. This resized region is denoted as I_{R_i}.

The feature extraction process uses a convolutional neural network (CNN), typically a pre-trained ResNet-50 network. Let f_{R_i} represent the feature vector extracted from region R_i using the CNN, which is passed through a fully connected layer for classification.

Mathematically, the feature extraction process can be written as:

$$f_{R_i} = \text{CNN}(I_{R_i})$$

where **CNN** is the feature extractor (ResNet-50 in this case).

After extracting the features, we use a fully connected layer to classify the object in each region proposal. The classification is performed by:

$$\hat{y}_i = \text{Softmax}(W f_{R_i} + b)$$

where:

- **W** is the weight matrix of the fully connected layer.
- **b** is the bias term.
- f_{R_i} is the feature vector for region R_i.
- $\hat{y}_i$ is the predicted class label for region R_i.

4.5 Selective Search Algorithm

Selective Search is a region proposal algorithm designed to generate object proposals for object detection tasks. The primary objective is to reduce the number of regions to be considered while maintaining the quality of proposed regions.

4.5.1 Algorithm Description

Given an image I, the selective search algorithm proceeds in several steps:

1. **Initial Segmentation:** The image is first divided into small regions (superpixels) using a segmentation algorithm like Simple Linear Iterative Clustering (SLIC). Let $S = \{S_1, S_2, \ldots, S_n\}$ represent the superpixels in the image I, where each S_i corresponds to a region in the image.
2. **Region Merging:** Regions S_i are iteratively merged based on similarity criteria such as color, texture, and size. This creates larger candidate regions. Let R_i denote a region formed after merging superpixels.

3. **Similarity Measure:** Similarity between two regions R_i and R_j is measured by:

$$\text{Sim}(R_i, R_j) = \alpha \cdot \text{Color}(R_i, R_j) + \beta \cdot \text{Texture}(R_i, R_j) + \gamma \cdot \text{Size}(R_i, R_j)$$

where:

- α, β, γ are weights controlling the importance of each similarity measure.
- **Color**(R_i, R_j) is the color similarity between regions R_i and R_j.
- **Texture**(R_i, R_j) is the texture similarity between regions R_i and R_j.
- **Size**(R_i, R_j) is the difference in size between the regions.

The algorithm merges regions with high similarity until a stopping criterion is met, such as a maximum number of regions or a minimum similarity threshold.

4. **Bounding Box Generation:** Once regions are merged, bounding boxes are computed for each merged region. The bounding box B_i for a region R_i is determined as:

$$B_i = (\min(x), \min(y), \max(x), \max(y))$$

where $\min(x), \min(y), \max(x), \max(y)$ correspond to the minimum and maximum pixel coordinates of the region.

4.5.2 Mathematical Formulation of Selective Search

Given an image I with n superpixels, the selective search algorithm iteratively merges these superpixels based on the similarity function and generates bounding box proposals. The final set of region proposals $R_1, R_2, \ldots, R_k$ represents candidate object locations in the image.

$$R = \{R_1, R_2, \ldots, R_k\}$$

where k is the number of regions generated by the selective search algorithm.

4.6 Training the Model

The model is trained using the following steps:

Let:

- $\hat{y}_i$ represent the predicted scores (logits) for the i-th region in the image.

- y_i represent the true label for the i-th region, where each region is associated with a corresponding class.
- N represent the total number of regions (sum of regions from all images in the batch).
- The predicted output $\hat{y}_i$ is a vector representing the raw scores from the model (before applying softmax), and the true labels y_i are class indices (integers).

The **Cross-Entropy Loss** function for a single region i is defined as:

$$\mathcal{L}_i = -\sum_{c=1}^{C} y_i^c \log(\hat{y}_{i,c})$$

where:

- C is the number of classes,
- y_i^c is the one-hot encoded true label for the i-th region, and
- $\hat{y}_{i,c}$ is the predicted probability of class c for the i-th region after applying the softmax transformation to the logits.

The total loss over all regions is the average of the individual region losses, which can be written as:

$$\mathcal{L}_{\text{total}} = \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}_i$$

This is the standard cross-entropy loss for classification problems, where the sum is taken over all regions in the image batch, and the loss is averaged over all regions.

In PyTorch, the function `torch.nn.CrossEntropyLoss()` automatically applies the softmax function to the logits before calculating the cross-entropy loss.

4.6.1 Steps in the Code:

1. **Prediction and Label Preparation**: The code gathers all predictions and target labels for all images in the batch. Each image may have a different number of regions, so the outputs are trimmed to match the number of regions.
2. **Concatenation**: The outputs and labels from all images in the batch are concatenated into single tensors: $\hat{Y}$ for predictions and Y for target labels.

3. **Cross-Entropy Loss Calculation**: The cross-entropy loss is computed between the concatenated predictions and target labels.

Thus, the overall loss function is a batch-wise computation of cross-entropy loss over all regions in all images:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} -\log \left(\frac{\exp(\hat{y}_i^{y_i})}{\sum_{c=1}^{C} \exp(\hat{y}_i^{c})} \right)$$

where:

- $\hat{y}_i^{y_i}$ is the predicted score for the true class label y_i,
- $\sum_{c=1}^{C} \exp(\hat{y}_i^{c})$ is the sum of exponentiated predicted logits (this is the normalization factor, i.e., the softmax operation).

This results in a scalar loss value that can be backpropagated to update the model's weights during training.

4.7 Optimization

The Adam optimizer is used to minimize the loss function. Adam is a popular optimization algorithm due to its adaptive learning rate, which adjusts based on the first and second moments of the gradients.

$$\theta_t = \theta_{t-1} - \eta \frac{m_t}{\sqrt{v_t} + \epsilon}$$

where: - η is the learning rate, - m_t and v_t are the first and second moments of the gradients, - ϵ is a small constant to avoid division by zero.

4.8 Model Evaluation

The model's performance is evaluated using several metrics, including precision, recall, F1-score, accuracy, and Intersection over Union (IoU).

4.8.1 Intersection over Union (IoU)

IoU is a measure of overlap between the predicted bounding box and the ground truth bounding box. It is calculated as:

$$IoU = \frac{\text{Area of Intersection}}{\text{Area of Union}}$$

where the area of intersection is the overlapping region between the predicted and true bounding boxes, and the area of union is the total area covered by both bounding boxes. The model is considered correct if the IoU exceeds a predefined threshold (e.g., 0.5).

4.8.2 Precision, Recall, and F1-Score

The precision, recall, and F1-score are calculated to evaluate the model's ability to correctly detect objects. These metrics are defined as follows:

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

4.8.3 Accuracy

Accuracy is the overall percentage of correct predictions (both true positives and true negatives) relative to the total number of predictions:

$$\text{Accuracy} = \frac{\text{True Positives} + \text{True Negatives}}{\text{Total Predictions}}$$


.png

Figure 11: Correct and Incorrect model prediction Examples

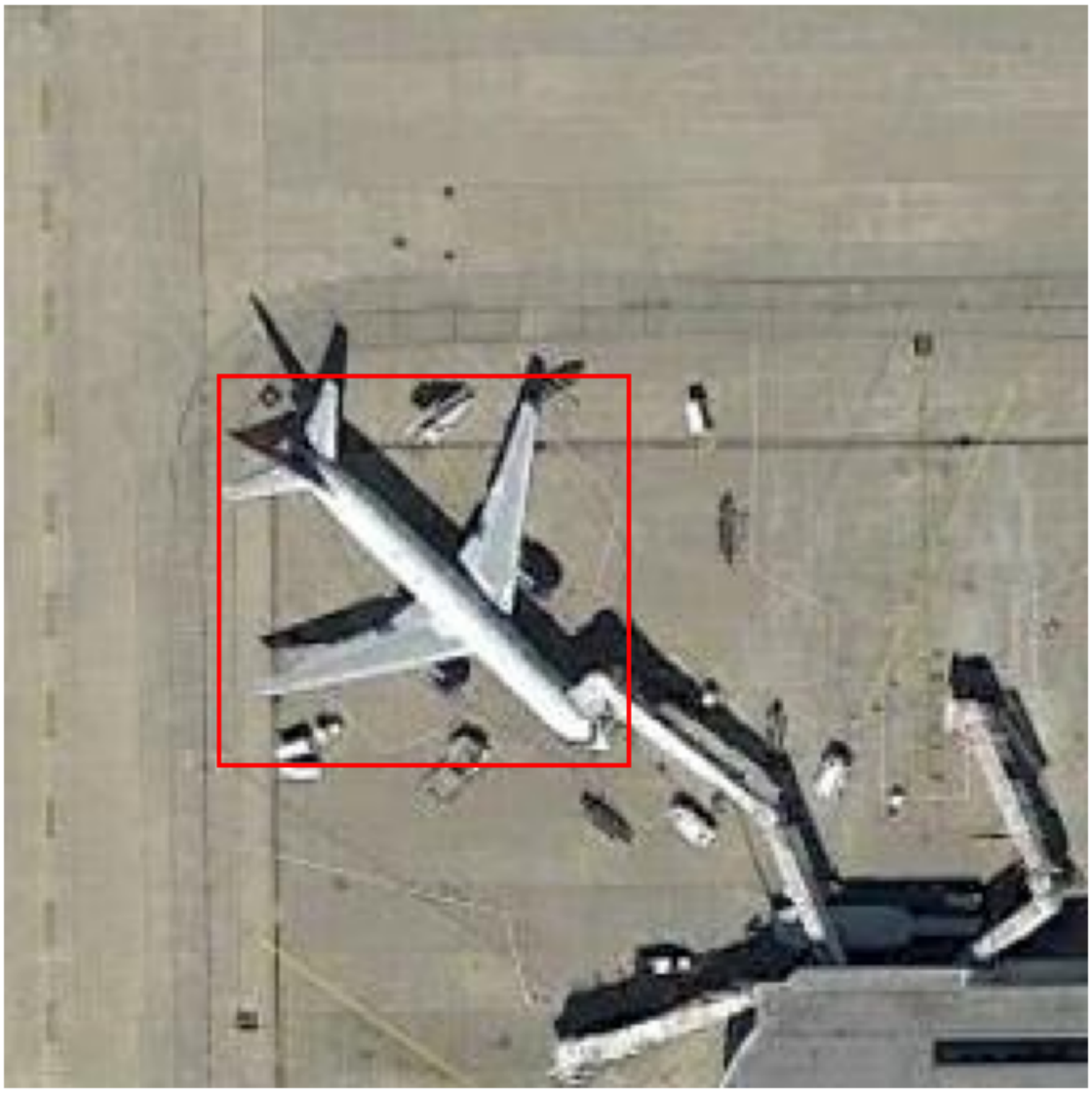
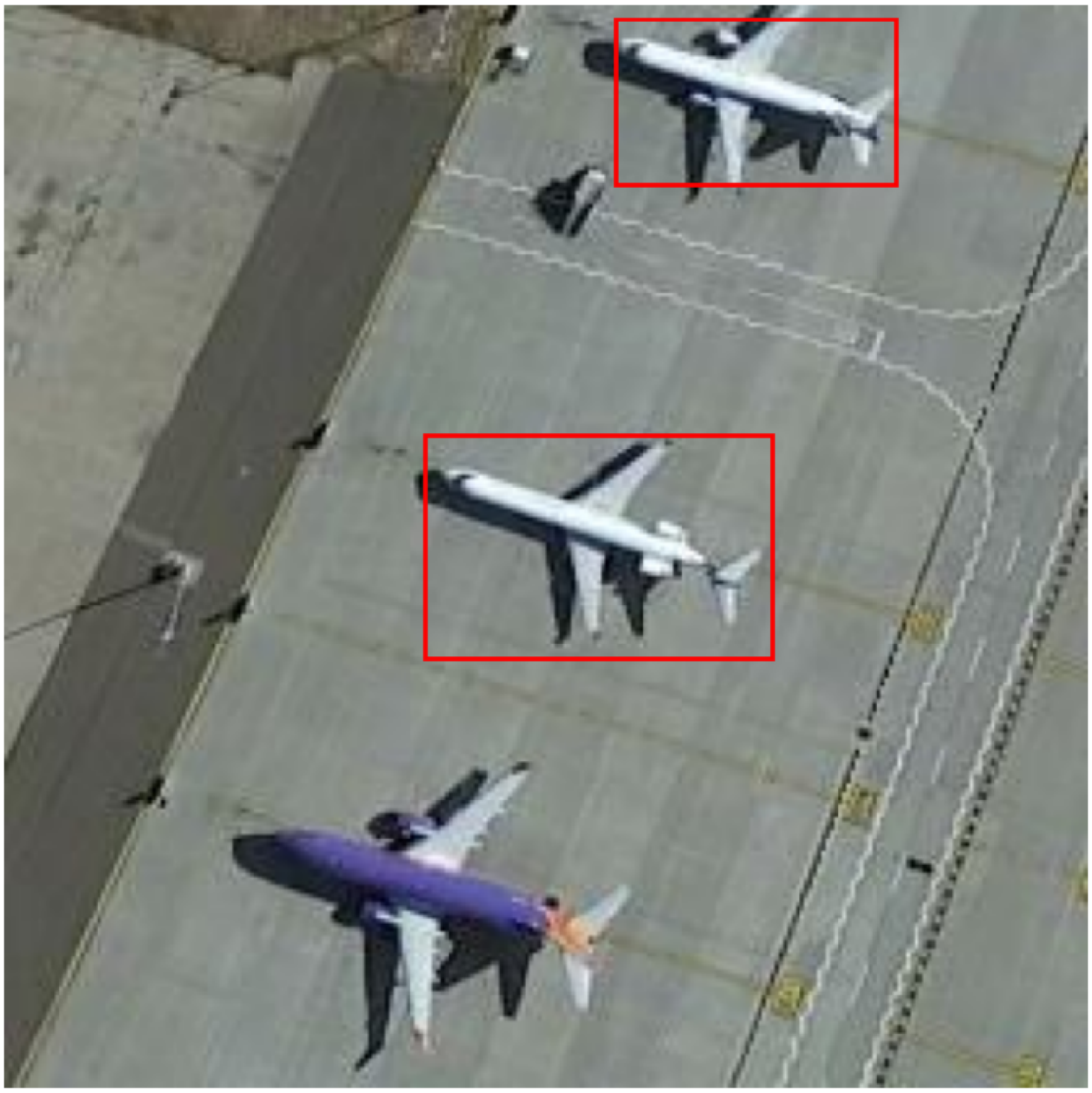

.png

Figure 12: Correct and Incorrect model prediction Examples



.png

Figure 13: Correct and Incorrect model prediction Examples

4.9 Performance Table

The table below summarizes the performance of the object detection model in terms of Average Precision (AP) at different IoU thresholds and the overall Mean Average Precision (MAP).

Metric	Value
AP@0.5	0.7281
AP@0.75	0.5905
MAP	0.6593

Table 10: Object Detection Evaluation Metrics

5 Q3. Not Selective Search Region-Based Convolution Neural Network

Algorithm 3 Simple Segmentation

0: **Input:** Image I of size (H, W, C)
0: **Output:** List of regions, where each region is represented by its bounding box coordinates
0: $I_{\text{cpu}} \leftarrow \text{cpu}(I)$
0: $I_{\text{numpy}} \leftarrow I_{\text{cpu}}.\text{numpy}()$
0: $I_{\text{reshaped}} \leftarrow I_{\text{numpy}}.\text{transpose}((1, 2, 0))$ {Reorder dimensions to (H, W, C)}
0: $I_{\text{scaled}} \leftarrow (I_{\text{reshaped}} \times 255).\text{astype}(\text{uint8})$ {Rescale pixel values to $[0, 255]$}
0: $S \leftarrow \text{SLIC}(I_{\text{scaled}}, n_{\text{segments}} = 100, \text{compactness} = 10, \sigma = 1)$ {Apply SLIC segmentation}
0: $R \leftarrow \emptyset$ {Initialize list to store bounding boxes}
0: **for** each unique label l in S **do**
0: $M_l \leftarrow (S == l)$ {Create mask for segment l}
0: $C_l \leftarrow \text{argwhere}(M_l)$ {Find coordinates of the segment}
0: $y_{\min}, x_{\min} \leftarrow \min(C_l)$ {Find minimum coordinates}
0: $y_{\max}, x_{\max} \leftarrow \max(C_l)$ {Find maximum coordinates}
0: $R \leftarrow R \cup \{(x_{\min}, y_{\min}, x_{\max}, y_{\max})\}$ {Add bounding box to list}
0: **end for**
0: **return** R {Return the list of bounding boxes for all regions} =0

5.1 Explanation of the Algorithm

The `simple_segmentation` algorithm performs segmentation of the input image using the Simple Linear Iterative Clustering (SLIC) method. The algorithm can be broken down into the following steps:

5.1.1 Preprocessing

The image I is first converted to a CPU-based numpy array, then reshaped and scaled to prepare it for the segmentation process. This is done to ensure that the input image has the appropriate format for processing by the SLIC algorithm. Specifically, the image is transposed to rearrange its dimensions from (C, H, W) to (H, W, C), and the pixel values are rescaled to the range $[0, 255]$.

$$I_{\text{scaled}} = (I_{\text{reshaped}} \times 255).\text{astype}(\text{uint8})$$

5.1.2 Segmentation

The Simple Linear Iterative Clustering (SLIC) method is applied to the preprocessed image. SLIC is a method for over-segmenting an image into superpixels, which are regions of the image that have similar color and spatial properties. The parameters for the SLIC method are set as $n_{\text{segments}} = 100$, compactness $= 10$, and $\sigma = 1$. These parameters control the number of segments, the strength of color versus spatial clustering, and the smoothing of color gradients, respectively.

$$S = \text{SLIC}(I_{\text{scaled}}, n_{\text{segments}} = 100, \text{compactness} = 10, \sigma = 1)$$

5.1.3 Region Extraction

For each unique segment l produced by the SLIC algorithm, the algorithm creates a binary mask M_l, which identifies the pixels belonging to that segment. The coordinates of the pixels within the segment are found using the `argwhere` function. The minimum and maximum x- and y-coordinates of the segment are then extracted, which form the bounding box for that segment. The bounding box coordinates $(x_{\min}, y_{\min}, x_{\max}, y_{\max})$ are added to the list of regions R.

$$M_l = (S == l) \quad \text{and} \quad C_l = \text{argwhere}(M_l)$$

Finally, the algorithm returns the list R containing the bounding box coordinates for all segments.


.png

Figure 14: Correct and Incorrect model prediction Examples

 .png

Figure 15: Correct and Incorrect model prediction Examples

5.2 Model Evaluation Results

The table below shows the evaluation metrics for the object detection model. These metrics include Average Precision (AP) at various Intersection over Union (IoU) thresholds and the overall mean Average Precision (mAP).

Metric	**Value**
AP@0.5	0.6112
AP@0.75	0.2829
MAP	0.4470

Table 11: Model Evaluation Results